

07 — Kịch bản quay video nội bài

Mới học thì đọc phần này là đủ. Phần chi tiết bên dưới chỉ để tra khi thấy hỏi sâu.

Bản rút gọn — 10 phút, 6 chặng

#	Mở cái gì	Nói đúng một ý
1	Database, chạy <code>SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES</code>	"Em có 10 bảng. Bảng <code>Orders</code> lưu đơn. Tiền em để <code>decimal</code> chứ không dùng <code>float</code> vì float bị sai số."
2	Landing, mở DevTools tab Network , điền form, bấm Xác nhận đặt hàng	"Bấm xong nó gửi <code>POST /api/orders</code> . Nhìn body này — không có giá tiền , client chỉ nói mua mấy cuốn."
3	<code>OrdersController.cs</code> dòng 18	"Request vào controller này. Controller không tính toán gì, nó gọi service."
4	<code>OrderService.cs</code> dòng 34–35 và 60	"Service đọc giá từ database rồi nhân số lượng. Xong gọi <code>SaveChangesAsync</code> , EF Core sinh câu <code>INSERT</code> ."
5	Database, <code>SELECT TOP 1 * FROM Orders ORDER BY Id DESC</code>	"Đây là đơn vừa đặt. $69.000 \times 3 = 207.000$, đúng cái server tính."
6	Admin: đăng nhập → vào trang đơn hàng	"Đăng nhập trả về JWT, trong token có <code>role</code> . Mọi request sau đều kèm token này."

Một câu chốt cuối:

"Điểm chính của bài là server không tin dữ liệu client gửi. Giá do server tính, định dạng do server kiểm, quyền do server chặn. Giao diện chỉ là lớp hiển thị."

Chứng minh nhanh (nên làm, 30 giây)

Dán lệnh này vào terminal, cố tình gửi giá bịa `totalPrice: 1`:

```
curl -X POST http://localhost:5080/api/orders -H "Content-Type: application/json" -d '{"custom
```

Kết quả vẫn ra **207000**. Nói: *"Em gửi giá 1 đồng mà server bỏ qua, nó tự tính lại."*

Thầy hỏi "sao đề 5 bảng mà em làm 10 bảng?"

Đây là câu dễ bị hỏi nhất. Trả lời đúng là **ăn điểm**, vì nó chứng tỏ hiểu chuẩn hoá.

Nói:

"Dạ trên bảng thầy vẽ 5 **nhóm chức năng**, em tách thành 10 bảng theo chuẩn 1NF ạ. Cụ thể nhóm *Thông tin Sách* có mục *Ảnh sách* và *Báo chí* — một cuốn sách có **nhiều** ảnh và **nhiều** trích dẫn báo chí. Nếu để chung một bảng thì em phải nhét danh sách vào một cột, kiểu 'Tuổi Trẻ|Thanh Niên|VnExpress' . Như vậy vi phạm 1NF: muốn sửa một trích dẫn thì phải cắt chuỗi, muốn đếm xem có mấy bài báo cũng không query được."

Rồi chiếu bảng đối chiếu này:

Nhóm trên bảng thầy	Bảng trong database	Quan hệ
Thông tin Sách	Books	bảng chính
→ Ảnh sách	BookImages	1-N — bắt buộc tách
→ Báo chí	PressQuotes	1-N — bắt buộc tách
→ Thông tin tác giả	Authors	1-1
→ File review	ContentReviews	1-1
Đơn đặt hàng	Orders	
Tài khoản / Role	AppUsers + Roles	Role là bảng tra cứu
Feedback	Feedbacks	
Setting	SiteSettings	

Nếu thầy hỏi tiếp "sao Authors với Roles cũng tách, 1-1 mà?":

"Dạ hai cái đó em tách cho dễ mở rộng ạ. Roles làm bảng riêng thì sau này thêm vai trò mới chỉ cần thêm một dòng, không phải sửa code. Còn Authors thì nếu sau này một cuốn có nhiều tác giả, em chỉ đổi khoá ngoại chứ không phải dựng lại bảng Books . Em biết là gộp vào Books cũng chạy được ạ."

Chiếu luôn sơ đồ khoá ngoại cho thầy thấy:

```
SELECT  fk.name AS KhoaNgoai,  
        OBJECT_NAME(fk.parent_object_id)      AS BangCon,  
        OBJECT_NAME(fk.referenced_object_id) AS BangCha  
FROM sys.foreign_keys fk ORDER BY BangCha, BangCon;
```

Chuẩn bị

```
$env:ASPNETCORE_ENVIRONMENT="Development"  
dotnet run --project backend/NhaGiaKim.Api --urls http://localhost:5080 --no-launch-profile  
# cửa sổ khác:  
npm --prefix frontend run dev
```

Mở sẵn: Chrome (DevTools Network) · VS Code · SSMS · terminal đang chạy `dotnet run`.

Phần chi tiết

Thầy chấm **luồng dữ liệu và hiểu biết kỹ thuật**, không chấm giao diện. Mỗi phần dưới đây ghi rõ: *mở file nào, dòng nào, nói gì, chiếu bằng chứng gì*.

Nguyên tắc xuyên suốt: mỗi lần nói "nó chạy qua đây" thì phải **chiếu thứ chứng minh** — dòng code, tab Network, hoặc record trong database. Nói miệng không có bằng chứng là mất điểm.

Chuẩn bị trước khi bấm ghi

```
cd D:/Documents/VHT-DT/BaiTapLTW

# Terminal 1 - backend. GIU NGUYEN cua so nay, cau SQL se in ra day.
$env:ASPNETCORE_ENVIRONMENT="Development"
dotnet run --project backend/NhaGiaKim.Api --urls http://localhost:5080 --no-launch-profile

# Terminal 2 - frontend
npm --prefix frontend run dev
```

Dọn bảng `Orders` cho sạch trước khi quay — chạy trong SSMS:

```
USE NhaGiaKim;
DELETE FROM Orders;
```

Mở sẵn 5 cửa sổ, sắp xếp để chuyển qua lại nhanh:

#	Cửa sổ	Dùng để
1	Chrome — http://localhost:5173 — mở sẵn DevTools tab Network	Thao tác và xem request
2	VS Code mở thư mục dự án	Chiếu code
3	Terminal đang chạy <code>dotnet run</code>	Chiếu câu SQL thật — EF Core in thẳng ra đây
4	SSMS nối <code>(localdb)\MSSQLLocalDB</code> bằng Windows Authentication	Chiếu bảng và record
5	Chrome tab 2 — http://localhost:5080/swagger	Chiếu hợp đồng API

Nói mở đầu (30 giây):

"Em làm website bán một đầu sách, kiến trúc tách đôi: backend là ASP.NET Core Web API, frontend là React gọi API qua HTTP. Em sẽ đi qua 3 luồng: đặt hàng của khách, đăng nhập admin, và hiển thị danh sách đơn. Mỗi luồng em đi từ lúc bấm nút cho tới khi dữ liệu nằm trong database."

Phần 1 — Luồng đặt hàng (khoảng 5 phút)

1.1 Chiếu database TRƯỚC khi đặt

Cửa sổ 4, chạy:

```
USE NhaGiaKim;
-- Thiết kế bảng
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE' ORDER BY TABLE_NAME;

-- Cấu trúc bảng Orders
SELECT COLUMN_NAME, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'Orders' ORDER BY ORDINAL_POSITION;

-- Số đơn hiện có
SELECT COUNT(*) AS SoDonTruocKhiDat FROM Orders;
```

Nói:

"Đây là 10 bảng trong database, đúng theo thiết kế ở docs/03-plan.md. Bảng Orders lưu đơn hàng. Chú ý cột UnitPrice và TotalPrice em để kiểu decimal(18,2) chứ không dùng float — tiền mà dùng số thực dấu phẩy động thì sẽ sai số khi cộng dồn. Hiện có N đơn."

1.2 Submit form, chiếu Network

Cửa sổ 1 → điền form đặt hàng → **mở Network trước khi bấm** → bấm "Xác nhận đặt hàng".

Click vào request orders trong Network, chiếu:

- **Headers:** POST http://localhost:5080/api/orders, Content-Type: application/json
- **Payload:** body JSON gửi lên
- **Response:** {"orderCode":"NGK- ...", "unitPrice":69000, "totalPrice":138000}, status 201 Created

Nói:

"Frontend không submit form theo kiểu POST truyền thống. React bắt sự kiện submit, gom dữ liệu thành JSON rồi gọi API bằng axios. Đây là payload gửi lên — chú ý **trong này không có giá tiền**, em sẽ giải thích vì sao ở phần sau."

1.3 Frontend: chỗ gửi request

File: `frontend/src/components/landing/OrderFormSection.tsx`

Dòng	Chiếu và nói
14–25	Schema Zod validate phía client: SĐT phải khớp <code>^0\d{9}\$</code> , số lượng 1–99
~47	<code>useMutation({ mutationFn: publicApi.createOrder, ... })</code> — chỗ gọi API
~56	<code>onSuccess</code> → <code>reset()</code> để F5 không gửi lại đơn

File: `frontend/src/api/endpoints.ts` → `publicApi.createOrder`

"Hàm này gọi `api.post('/api/orders', payload)`. `api` là instance axios cấu hình ở `frontend/src/api/client.ts`."

Điểm ăn điểm — nói rõ:

"Em validate ở cả hai phía. Nhưng validate ở client **chỉ để trải nghiệm tốt**, nó không phải bảo mật — ai cũng mở DevTools sửa được. Nguồn chân lý là backend, em sẽ chứng minh ngay."

1.4 Backend: Controller nhận request

File: `backend/NhaGiaKim.Api/Controllers/OrdersController.cs`

Dòng	Nội dung
9	<code>[Route("api/orders")]</code> — đường dẫn khớp đúng cái vừa thấy trong Network
13	<code>[HttpPost]</code>
14	<code>[EnableRateLimiting(RateLimitPolicies.PublicWrite)]</code> — chống spam 5 đơn/phút/IP
18	<code>Create([FromBody] CreateOrderRequest request, ...)</code> → gọi <code>orderService.CreateAsync</code>

Nói:

"Controller **không chứa logic nghiệp vụ**. Nhiệm vụ của nó chỉ là nhận HTTP, giao cho service, rồi đổi kết quả thành mã HTTP. Em tách như vậy để logic test được mà không cần dựng web server."

Chỗ validate của backend:

backend/NhaGiaKim.Application/Validators/CreateOrderRequestValidator.cs

"FluentValidation chạy tự động trước khi vào controller. Sai định dạng là trả 400, không bao giờ chạm tới service."

1.5 Service: xử lý nghiệp vụ

File: backend/NhaGiaKim.Application/Services/OrderService.cs

Dòng	Chiều và nói
15	<code>OrderService(IAppDbContext db, TimeProvider clock, IDbExceptionClassifier dbErrors)</code> — DI qua constructor
23–32	Truy vấn sách đang mở bán; không có thì trả <code>NotFound</code>
34–35	<code>MoneyCalculator.EffectiveUnitPrice(book.Price, book.DiscountPrice)</code> rồi <code>Total(unitPrice, quantity)</code>
42	<code>OrderCode = OrderCodeGenerator.Generate(now)</code>
57	<code>db.Orders.Add(order)</code>
60	<code>await db.SaveChangesAsync(ct)</code> ← đây là lúc EF sinh câu <code>INSERT</code>
61	Trả về <code>ServiceResult.Ok(new CreateOrderResponse(...))</code>

Đoạn này nhấn mạnh (dòng 34–35):

"Giá **không** lấy từ dữ liệu client gửi lên. Server đọc giá từ bảng `Books` rồi nhân với số lượng. Nếu không làm vậy, khách sửa JSON trong DevTools là mua được giá 1 đồng. Em có test chứng minh điều này ở `PublicApiTests.cs`, tên test là `CreateOrder_IgnoresClientSuppliedTotal`."

Dòng 64–72 — xử lý trùng mã đơn:

"Mã đơn sinh ngẫu nhiên nên có thể trùng. Cột `OrderCode` có unique index, nên nếu trùng thì `SaveChangesAsync` ném `DbUpdateException`. Em bắt đúng loại lỗi trùng khoá thôi rồi sinh mã mới, thử tối đa 3 lần. Các lỗi khác như mất kết nối thì để nó nổi lên thành 500, không nuốt."

1.6 Chiếu câu SQL thật

Cửa sổ 3 (terminal đang chạy `dotnet run`) — chiếu câu `INSERT` EF Core vừa sinh:

```
INSERT INTO [Orders] ([Address], [BookId], [CreatedAt], [CustomerName], [Note],
                      [OrderCode], [PaymentMethod], [Phone], [Quantity], [Status],
                      [TotalPrice], [UnitPrice], [UpdatedAt])
OUTPUT INSERTED.[Id]
VALUES (@p0, @p1, @p2, ...);
```

Nói:

"Em không viết SQL tay. EF Core dịch `db.Orders.Add()` + `SaveChangesAsync()` thành câu này. Chú ý nó dùng **tham số @p0, @p1** chứ không nối chuỗi — nhờ vậy không bị SQL injection."

1.7 Chiếu record trong database

Cửa sổ 4:

```
SELECT TOP 1 Id, OrderCode, CustomerName, Phone, Quantity, UnitPrice, TotalPrice,
           PaymentMethod, Status, CreatedAt
FROM Orders ORDER BY Id DESC;
```

Nói:

"Đơn vừa đặt đã nằm trong database. `UnitPrice` 69.000, `Quantity` 2, `TotalPrice` 138.000 — đúng bằng server tính. `Status` = 0 là trạng thái `New`, em lưu enum dưới dạng số."

1.8 Chứng minh backend là nguồn chân lý (nên có — rất dễ ghi điểm)

Mở Swagger (cửa sổ 5) hoặc dùng terminal, gửi request **cố tình gửi kèm giá bịa**:

```
curl -X POST http://localhost:5080/api/orders -H "Content-Type: application/json" -d '{"custom
```

Nói:

"Em gửi kèm `totalPrice: 1`. Kết quả trả về vẫn là 138.000 — server bỏ qua hoàn toàn giá client gửi."

Rồi gửi SĐT sai để thấy 400:

```
curl -i -X POST http://localhost:5080/api/orders -H "Content-Type: application/json" -d '{"cus
```

"400 kèm `errors.Phone`, và **không có record nào được tạo** — em có test đếm số dòng trước/sau để chứng minh, tên test `CreateOrder_InvalidPhone_WritesNothingToDatabase`."

Phần 2 — Luồng đăng nhập admin (khoảng 4 phút)

2.1 Submit form đăng nhập

Vào <http://localhost:5173/admin/login>, mở Network, đăng nhập.

Chiếu request `login`:

- `POST /api/auth/login`, payload `{"email": "...", "password": "..."}"`
- Response: `{"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50a3V5In0: "...", "expiresAtUtc": "...", "user": {"id": 1, "role": "Admin"}"}`

Nói:

"Giống phần đặt hàng, React gom email và mật khẩu rồi POST lên API. Mật khẩu đi trong body chứ không nằm trên URL."

2.2 Frontend: nơi gửi và nơi lưu token

File: `frontend/src/pages/admin/LoginPage.tsx` → gọi `login()` của `useAuth` **File:** `frontend/src/hooks/useAuth.tsx` dòng 37–41

```
const result = await authApi.login(email, password)
localStorage.setItem(TOKEN_STORAGE_KEY, result.accessToken)
setUser(result.user)
```

File: `frontend/src/api/client.ts` — interceptor tự gắn token vào MỌI request sau đó:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_STORAGE_KEY)
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

Nói:

"Em lưu token vào `localStorage`, rồi interceptor của axios tự gắn header `Authorization: Bearer` cho mọi request sau. Nhờ vậy không phải nhớ gắn token ở từng chỗ gọi API."

2.3 Backend: Controller → Service

File: `backend/NhaGiaKim.Api/Controllers/AuthController.cs` dòng 9, 12, 16–17

File: `backend/NhaGiaKim.Application/Services/AuthService.cs`

Dòng	Chiều và nói
33–35	<code>db.AppUsers.Include(u => u.Role).FirstOrDefaultAsync(u => u.Email == email, ct)</code>
39–40	<code>hashToCheck = user is { IsActive: true } ? user.PasswordHash :</code> <code>hasher.DummyHash</code> rồi <code>hasher.Verify(...)</code>
42–45	Sai thì trả 401 với thông báo chung chung
48	<code>jwt.Generate(user, roleName)</code>
50	Cập nhật <code>LastLoginAt</code>

Hai điểm ăn điểm, nói kỹ:

Thứ nhất — mật khẩu không bao giờ lưu dạng gốc. Trong bảng `AppUsers` chỉ có `PasswordHash`. Em dùng BCrypt với work factor 12. BCrypt là hàm một chiều, có salt riêng cho từng mật khẩu, nên hai người đặt mật khẩu giống nhau vẫn ra hash khác nhau.

Chiều luôn trong database:

```
SELECT Id, Email, LEFT>PasswordHash, 35) + '...' AS Hash, IsActive, RoleId FROM AppUsers;
```

"Thấy không, nó bắt đầu bằng `$2a$12$` — đó là định dạng BCrypt, số 12 là work factor."

Thứ hai — chống dò email. Dòng 39: kể cả khi email **không tồn tại**, em vẫn chạy một phép BCrypt verify với một hash giả. Nếu không làm vậy, email sai trả về sau 2 mili giây còn email đúng mà sai mật khẩu mất 300 mili giây — kẻ tấn công đo thời gian là biết email nào có thật. Thông báo lỗi cũng để chung chung "Email hoặc mật khẩu không đúng", không nói rõ sai cái nào.

2.4 Sinh JWT và gán role

File: backend/NhaGiaKim.Infrastructure/Security/JwtTokenGenerator.cs dòng 20–37

```
Claim[] claims =  
[  
    new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),  
    new(JwtRegisteredClaimNames.Email, user.Email),  
    new(ClaimTypes.Role, roleName),    // ← chỗ gán role  
];  
...  
signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256)
```

Nói:

"Token chứa các claim: id người dùng, email, và **role**. Token được ký bằng HMAC-SHA256 với khoá bí mật nằm trong biến môi trường, không nằm trong code. Ai sửa nội dung token thì chữ ký không khớp, server từ chối ngay."

Chứng minh trực quan: copy token từ Network, dán vào <https://jwt.io>

"Phần payload giải mã ra thấy `role: Admin`. Lưu ý JWT chỉ **mã hoá base64** chứ không **mã hoá bí mật** — ai cũng đọc được nội dung. Cái bảo vệ nó là chữ ký, không phải sự che giấu. Nên em không bao giờ bỏ mật khẩu vào token."

2.5 Role được dùng để chặn quyền ở đâu

File: backend/NhaGiaKim.Api/Program.cs dòng 69–70

```
.AddPolicy(AuthPolicies.RequireAdmin, p => p.RequireRole(RoleName.Admin))  
.AddPolicy(AuthPolicies.RequireStaffOrAdmin, p => p.RequireRole(RoleName.Admin, RoleName.Staff))
```

File: backend/NhaGiaKim.Api/Controllers/Admin/AdminOrdersController.cs dòng 12

```
[Authorize(Policy = AuthPolicies.RequireStaffOrAdmin)]
```

Nói:

"Em khai 2 policy. Trang đơn hàng cho cả Staff và Admin; còn sửa thông tin sách thì chỉ Admin. Em sẽ chứng minh bằng cách đăng nhập tài khoản Staff."

Chứng minh: đăng nhập Staff → thấy menu ít mục hơn → gọi API admin bị chặn:

```
curl -i -X PUT http://localhost:5080/api/admin/book -H "Authorization: Bearer <TOKEN_STAFF>" -H
```

"403 Forbidden. Quan trọng: em **không chỉ ẩn menu ở frontend**. Ẩn menu chỉ là cho gọn mắt, ai cũng gõ thẳng URL hoặc gọi API được. Chặn thật nằm ở server."

Điểm cộng — nói thêm nếu còn thời gian

(backend/NhaGiaKim.Api/Security/ActiveUserValidator.cs):

"JWT là stateless, phát ra rồi thì không thu hồi được cho tới khi hết hạn 60 phút. Nghĩa là admin khoá một tài khoản xong, người đó vẫn vào được 60 phút nữa. Em vá bằng cách mỗi request đã xác thực thì đối chiếu lại IsActive và role với database. Trả giá là thêm một truy vấn nhỏ mỗi request, nhưng với khu quản trị thì đáng."

Phần 3 — Danh sách đơn hàng trong admin (khoảng 4 phút)

3.1 Load trang, chiếu Network

Vào <http://localhost:5173/admin/orders>. Trong Network chiếu request:

```
GET /api/admin/orders?page=1&pageSize=10
Request Headers: Authorization: Bearer eyJ...
```

Nói:

"Token được interceptor tự gắn vào đây. Response là JSON có items, totalCount, page, pageSize — em phân trang ở **server** chứ không tải hết về rồi cắt ở client. Nếu có 10.000 đơn mà tải hết thì trình duyệt treo."

3.2 Frontend: nơi gọi và nơi đổ dữ liệu ra bảng

File: frontend/src/pages/admin/OrdersPage.tsx

Chiếu	Nói
<code>useQuery({ queryKey: ['admin', 'orders', filter], queryFn: () => adminApi.getOrders(filter) })</code>	"Em dùng thư viện TanStack Query để gọi API. Nó lo cache, trạng thái đang tải, và tự gọi lại khi filter đổi."
<code>const columns = [...]</code>	"Đây là định nghĩa cột cho bảng."
<code><Table columns={columns} dataSource={rows} pagination={...} /></code>	"Bảng em dùng component Table của Ant Design ."

Thầy hỏi 'dùng thư viện JS nào' — trả lời rõ:

"Frontend em dùng **React 19**, bảng và form dùng **Ant Design 6**, gọi API bằng **axios**, quản lý trạng thái dữ liệu server bằng **TanStack Query**, validate form bằng **Zod** kết hợp **react-hook-form**. Giao diện landing dùng **TailwindCSS**."

3.3 Backend: filter dịch thành SQL ra sao

File: `backend/NhaGiaKim.Application/Services/AdminOrderService.cs`

Dòng	Nội dung
32	<code>q = q.Where(o => o.Status == query.Status.Value)</code>
37	<code>q = q.Where(o => o.Phone.Contains(phone))</code>
41, 47	Lọc theo khoảng ngày
50	<code>var total = await q.CountAsync(ct)</code>
52-54	<code>.OrderByDescending(o => o.CreatedAt).Skip((page-1)*pageSize).Take(pageSize)</code>

Nói:

"Mỗi điều kiện lọc là một `.Where()` cộng dồn vào cùng một `IQueryable`. Quan trọng: **tới đây chưa có truy vấn nào chạy**. EF chỉ dựng cây biểu thức. Khi gặp `CountAsync` và `ToListAsync` nó mới dịch tất cả thành một câu SQL rồi gửi xuống. Nhờ vậy lọc và phân trang làm ở database, không phải tải hết về rồi lọc trong bộ nhớ."

Chiếu SQL thật (cửa sổ 3) — thao tác lọc trên UI rồi xem log:

```
SELECT [o].[Id], [o].[OrderCode], ... FROM [Orders] AS [o]
WHERE [o].[Status] = @__status_0 AND ([o].[Phone] LIKE @__phone_1)
ORDER BY [o].[CreatedAt] DESC
OFFSET @__p_2 ROWS FETCH NEXT @__p_3 ROWS ONLY;
```

"OFFSET ... FETCH NEXT chính là skip / Take em viết bằng LINQ."

3.4 Đổi trạng thái đơn — luật nghiệp vụ

Bấm "Đã xác nhận" trên một đơn, chiều Network: `PATCH /api/admin/orders/{id}/status`.

File: `backend/NhaGiaKim.Application/Common/OrderStateMachine.cs`

```
[OrderStatus.New]          = [OrderStatus.Confirmed, OrderStatus.Cancelled],  
[OrderStatus.Confirmed]    = [OrderStatus.Shipping, OrderStatus.Cancelled],  
[OrderStatus.Shipping]     = [OrderStatus.Completed],  
[OrderStatus.Completed]    = [],
```

File: `AdminOrderService.cs` dòng 78 — `if (!OrderStateMachine.CanTransition(...)) return Invalid(...)`

Nói:

"Trạng thái đơn không được nhảy lung tung. Em mô hình hoá thành máy trạng thái: đơn Mới chỉ được sang Đã xác nhận hoặc Huỷ. Giao diện chỉ hiện nút hợp lệ, **nhưng server vẫn kiểm lại** — vì ai cũng gọi thẳng API được."

Chứng minh:

```
curl -i -X PATCH http://localhost:5080/api/admin/orders/1/status -H "Authorization: Bearer <TOK
```

"Đơn đang ở Đã xác nhận mà em ép về Mới — server trả 400 kèm lý do."

Chiếu database để thấy `UpdatedAt` đổi:

```
SELECT Id, OrderCode, Status, CreatedAt, UpdatedAt FROM Orders ORDER BY Id DESC;
```

Phần 4 — Chốt lại (khoảng 2 phút)

4.1 Kiến trúc phân tầng

Mở `docs/03-plan.md` phần 1, chiếu sơ đồ:

```
React SPA → ASP.NET Core Web API → SQL Server  
          Controller → Service → ApplicationDbContext (EF Core)
```

Nói:

"Backend em chia 4 project: `Domain` chứa entity thuần không phụ thuộc gì; `Application` chứa logic nghiệp vụ và service; `Infrastructure` lo EF Core, BCrypt, JWT; `Api` chỉ có controller và cấu hình. Luật là tầng trong không biết tầng ngoài — nhờ vậy đổi từ SQL Server sang database khác thì chỉ sửa `Infrastructure`."

4.2 Bảng chứng kiểm thử

```
bash scripts/gates.sh
```

"Em có 132 test tự động: 107 test backend gồm unit test cho luật nghiệp vụ và integration test gọi API thật, 25 test frontend. Thêm 12 test E2E chạy trên trình duyệt thật bằng Playwright. Mỗi yêu cầu trong `docs/01-spec.md` đều truy vết được tới ít nhất một test, bảng truy vết ở `docs/02-test-design.md`."

Mở `docs/05-review-findings.md` :

"Em còn cho review đọc lập soát lại và tìm ra 3 lỗi chặn, 8 lỗi nặng — ví dụ chữ trắng trên nền vàng chỉ đạt tương phản 2.64:1, trượt chuẩn WCAG. Đã sửa hết và ghi lại lý do từng cái."

Những chỗ cần nói thật, đừng né

Thầy hỏi những thứ dự án này **không có**. Trả lời thẳng sẽ tốt hơn là lờ đi:

Thầy hỏi	Sự thật	Nên nói
Biểu đồ dùng thư viện JS nào	Dự án không có biểu đồ	"Dạ bài này em chưa làm biểu đồ vì phạm vi chỉ bán một đầu sách, thống kê chưa đủ nhiều để vẽ. Nếu thêm thì em sẽ dùng Recharts, gọi một endpoint thống kê riêng rồi đổ vào component biểu đồ."
Đoạn JS xử lý form submit	Không phải JS thuần hay jQuery, mà là React	"Em không dùng jQuery. React bắt sự kiện submit qua react-hook-form, validate bằng Zod rồi gọi axios. Em chiếu đúng chỗ đó trong <code>OrderFormSection.tsx</code> ."
Application context	Dùng DI container của ASP.NET Core	"Tương đương Application Context là <code>IServiceCollection</code> . Em đăng ký service ở <code>Infrastructure/DependencyInjection.cs</code> , rồi constructor của controller và service tự nhận vào."

Nếu muốn thêm biểu đồ trước khi nộp thì nói, mình làm — cần thêm một endpoint thống kê (doanh thu theo ngày, số đơn theo trạng thái) và một trang Dashboard.

Mẹo quay

- **Phóng to chữ VS Code lên cỡ 16–18** trước khi quay, không thấy phải căng mắt.
- Mỗi lần chuyển file thì **đọc to đường dẫn** ("em mở file `OrderService.cs` trong thư mục Application").
- Nói **vì sao** chứ không chỉ nói **cái gì**. "Em để giá tính ở server" là mô tả; "nếu tính ở client thì khách sửa DevTools là mua giá 1 đồng" mới là hiểu.
- Dừng ở mỗi bằng chứng 2–3 giây cho người xem kịp đọc.
- Tổng khoảng **15 phút**. Dài quá thầy không xem hết.